Local development

A `docker-compose.yml` at the repo root runs the whole pipeline on one machine: the framework's own services (`webhook-listener`, `dispatch-worker`) and a local Temporal server. A story entering `In Progress` starts a real Temporal workflow, which launches a real `specialist-runner` container on your Docker engine.

By default it runs the published images (`ghcr.io/kisasa/intent-to-production/...`), so nothing is built on your machine. To run your own changes, see "Running your own changes" below.

What's real, what's local:

Piece	Local dev
Linear, GitHub, Anthropic	**Real, external.** Nothing here emulates any of the three — real credentials, real hosted APIs.
Temporal	**Local.** `temporalio/auto-setup` + Postgres + `temporal-ui` — no TLS, no namespace API key.
Specialist containers	**Local**, on the same Docker engine compose uses. `dispatch-worker` launches one per dispatch through the engine socket mounted into it (`dispatch-worker/src/launchers/docker-launcher.ts`).

Prerequisites

- Docker Desktop, or another Docker engine with Compose, running.
- A real Linear workspace (a sandbox one is fine), a real GitHub org/repo you can write branches and PRs to, and real API keys for Linear, GitHub, and Anthropic.
- cloudflared (or any tunnel tool), so Linear's webhooks can reach your machine — see `webhook-listener/README.md`.

Bring-up order

1. **Copy `docker-compose.override.yml.example` to `docker-compose.override.yml` and fill in your real secret values.** Every env var each service needs is listed in `docker-compose.yml` itself, with working defaults for everything non-secret and empty placeholders for the secrets. `docker compose` merges `docker-compose.override.yml` automatically (gitignored — never commit your filled-in copy), key by key, so it only names what it sets.

`FRAMEWORK_REPO` on `dispatch-worker` names the repository (`org/name`) the specialist clones its agent definition and skills from. It defaults to this project's own; set it to your fork only if you have changed them, with `FRAMEWORK_REF` (default `main`) naming the branch.

`dispatch-worker` passes its own `ANTHROPIC_API_KEY`, `GITHUB_TOKEN`, `LINEAR_AGENT_API_KEY`, `FRAMEWORK_REPO`, `FRAMEWORK_REF`, `CLAUDE_MODEL` and `CLAUDE_EFFORT` to every specialist it launches (`SPECIALIST_FORWARD_ENV` in `docker-compose.yml`). If any of them is empty, the worker refuses to start

and names it.

To have the app request you as the PR's reviewer, also set `REVIEWER_EMAIL_TO_GITHUB_LOGIN` on `dispatch-worker`, a JSON object mapping your Linear email to your GitHub login (e.g. `{"you@example.com":"your-github-login"}`). Without an entry for whoever moves the story, dispatch still works, but no reviewer is requested. Revision rounds do not need it: a "request changes" review from any account with write access to the repository starts one.

The per-package `.env.example` files (`webhook-listener/`, `dispatch-worker/`, `specialist-runner/`) are for running one package standalone outside Docker (each README's own "Run it locally" section) or, for `specialist-runner`, testing the image by hand. Compose does not read them.

1 **Bring up the stack:**

``bash docker compose up ``

This pulls the three images at `ITP_VERSION` (default `latest`, the newest release; `main` for the newest commit, or a release such as `0.1.0`). The `specialist-runner` service only pulls the specialist image and exits — `docker compose ps` shows it `Exited (0)`, and `dispatch-worker` waits for that before starting. `docker compose pull` fetches a newer image under the same tag.

1 **Tunnel a real Linear webhook in:**

``bash cloudflared tunnel --url http://localhost:8787 ``

Put the resulting URL (`https://…/webhooks/linear`) into your Linear workspace's webhook settings (Settings → API → Webhooks).

Triggering and watching a dispatch

Before a status move can dispatch anything, the tracker and repo need what a real engagement has:

- **A `Surfaces` document on the project**, with a record for each surface the story is labelled with (repo, ref, and optionally path, conventions file, mandatory skills). Without a matching record, dispatch fails and the story goes back to Todo with a comment.
- **A story carrying a `surface:<name>` label** that matches a record. The lane ignores a story with no `surface:` label.
- **The epic's branch in the target repo**, named as the tracker names it and cut from a BRD branch. The app creates the story branch; it does not create the epic branch.
- **A `CONVENTIONS.md`** at the surface root on that ref. The specialist stops without one.
- **A `Blocking dependencies` section** in the story's description ("No blocking dependencies." is fine). Every story it names must be Done.

Then move the story to the status named `In Progress`; that exact name is what the lane matches. Watch it happen:

- **Temporal UI** — `http://localhost:8080` — the workflow execution, its activities, and the heartbeat detail of the one waiting on the specialist.
- `docker compose logs -f webhook-listener dispatch-worker` — the same structured logs described in each package's own README.

- **The specialist itself** — `docker ps --filter label=itp.role=specialist` shows its container once dispatched, and `docker logs -f <name>` follows it. Exited specialists are kept for their logs; clear them with `docker container prune --filter label=itp.role=specialist`.

Running your own changes

`docker-compose.build.yml` builds all three images from your checkout and tags them `:local`. Naming any compose file stops `docker compose` from merging the override on its own, so name all three:

```
docker compose -f docker-compose.yml -f docker-compose.build.yml -f docker-compose.override.yml up --build
```

Why `specialist-runner` isn't a running service

It's ephemeral by design — one container per dispatch, started by `dispatch-worker`, never a long-running process. The worker runs whatever image `SPECIALIST_IMAGE` names from the local image cache, pulling it only if it is missing. When building locally, it does not rebuild it either: after any change under `specialist-runner/`, run the build command above again before triggering another dispatch, or the old image runs.

The Docker socket

`dispatch-worker` mounts `/var/run/docker.sock` and runs as root so it can open it. Access to that socket is root-equivalent on the host, which is the price of launching containers from inside one. That is fine on a development machine; anywhere else, run the worker where that access is acceptable, or give it a different launcher (see `dispatch-worker/README.md`, "Specialist launcher").

What this does NOT do

- No live-reload — with the build file, each service rebuilds from its Dockerfile on `up --build`. No volume-mounted source, no `nodemon`.
- No mocked Linear/GitHub/Anthropic — a real workspace, real repo, and real API keys are required to drive anything past `webhook-listener` accepting a webhook.
- No deployment. Running this anywhere but your own machine is a deployment decision the framework leaves to you: it needs the listener as a single always-on instance, a Temporal server, the worker, and somewhere for the worker's launcher to start specialist containers.